SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Week 5: Task C.2

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY – FEB-2026

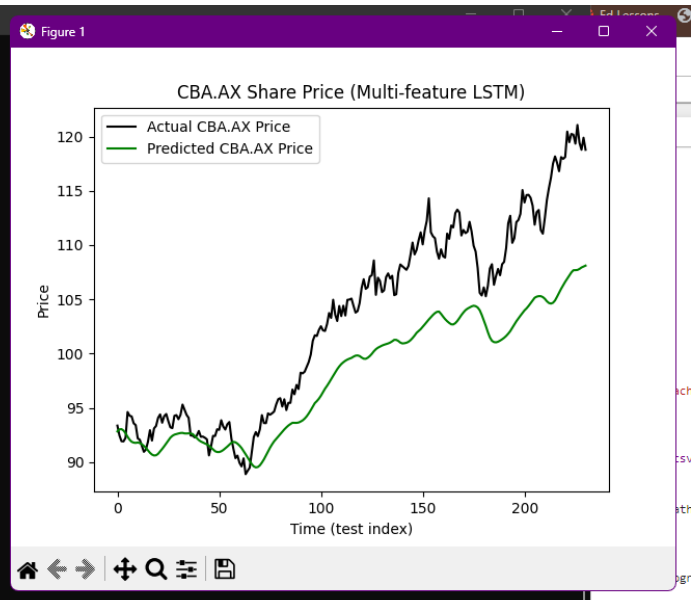
Contents

Evidence of execution (from my notebook output) 2
1) What I implemented (overview) 3
2) Execution evidence (what I observed) 4
3) Explanation of less straightforward lines of code (with reasoning) 5
3.1 Local caching (save/load CSV to reduce repeated downloads) 5
3.2 Flattening yfinance MultiIndex columns (critical in some environments) 6
3.3 NaN handling strategies (why it's done before scaling and sequencing) 6
3.4 Split methods (date / ratio / random) and preventing time leakage 7
3.5 Scaling features correctly (fit on train only + store scalers) 8
3.6 Building LSTM sequences (3D shape) 9

Evidence of execution (from my notebook output)

```
Dataset range: 2020-01-01 → 2024-07-02
Split method used: date
Loaded from cache: data_cache\CBA.AX_2020-01-01_2024-07-02.csv
NaN before: 0
NaN after : 0
[Split=date] X_train=(846, 60, 5), X_test=(231, 60, 5)
Loaded from cache: data_cache\CBA.AX_2020-01-01_2024-07-02.csv
NaN before: 0
NaN after : 0
[Split=ratio] X_train=(849, 60, 5), X_test=(228, 60, 5)
Loaded from cache: data_cache\CBA.AX_2020-01-01_2024-07-02.csv
NaN before: 0
NaN after : 0
[Split=random] X_train=(861, 60, 5), X_test=(216, 60, 5)
Loaded from cache: data_cache\CBA.AX_2020-01-01_2024-07-02.csv
NaN before: 0
NaN after : 0
Raw df shape: (1137, 5)
Clean df shape: (1137, 5)
Train/Test samples: 846 231
X_train shape (samples, lookback, n_features): (846, 60, 5)
Scaler keys: ['feature_scaler', 'target_scaler', 'feature_columns', 'target_column']
Feature columns: ['Open', 'High', 'Low', 'Close', 'Volume']
Target column: Close
```

```
Epoch 12/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 0.0062
Epoch 13/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 0.0064
Epoch 14/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 0.0061
Epoch 15/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 0.0053
Epoch 16/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 0.0057
Epoch 17/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 0.0065
Epoch 18/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 0.0056
Epoch 19/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 0.0055
Epoch 20/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 0.0056
Epoch 21/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - loss: 0.0055
Epoch 22/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - loss: 0.0051
Epoch 23/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 24ms/step - loss: 0.0054
Epoch 24/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 0.0054
Epoch 25/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 23ms/step - loss: 0.0050
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
```




```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
Next-day prediction for CBA.AX: [[108.17087]]
```

1) What I implemented (overview)

The starter code v0.1 had two major limitations: (1) the user had to manually choose separate date ranges for training and test downloads, and (2) the model ignored most features and used

Close only. For Task C.2, I implemented a reusable data-loading and processing function that supports:

- Whole dataset date range as inputs (single download for the full dataset period)
- NaN handling strategies (fill/drop/interpolate)
- Multiple splitting methods (date / ratio / random)
- Local caching (save downloaded CSV and reload later to save time)
- Scaling for all feature columns and storing scalers in a dictionary for later inverse-transform of predictions

I validated the implementation using CBA.AX and printed execution evidence including dataset range, NaN counts, split shapes, cache loading, and scaler keys.

2) Execution evidence (what I observed)

When running the script, I obtained the following evidence:

- Dataset range: 2020-01-01 → 2024-07-02
- Cache behavior: Loaded from cache: data_cache\CBA.AX_2020-01-01_2024-07-02.csv
- NaN handling: NaN before: 0; NaN after: 0
- Split methods demonstrated:
    - Date split: X_train = (846, 60, 5), X_test = (231, 60, 5)
    - Ratio split: X_train = (849, 60, 5), X_test = (228, 60, 5)
    - Random split: X_train = (861, 60, 5), X_test = (216, 60, 5)
- Scaler structure stored:
- ['feature_scaler', 'target_scaler', 'feature_columns', 'target_column']
- Feature set used: Open, High, Low, Close, Volume
- Model input shape: (samples, lookback, n_features) = (846, 60, 5).

```
Dataset range: 2020-01-01 → 2024-07-02
Split method used: date
Loaded from cache: data_cache\CBA.AX_2020-01-01_2024-07-02.csv
NaN before: 0
NaN after : 0
[Split=date] X_train=(846, 60, 5), X_test=(231, 60, 5)
Loaded from cache: data_cache\CBA.AX_2020-01-01_2024-07-02.csv
NaN before: 0
NaN after : 0
[Split=ratio] X_train=(849, 60, 5), X_test=(228, 60, 5)
Loaded from cache: data_cache\CBA.AX_2020-01-01_2024-07-02.csv
NaN before: 0
NaN after : 0
[Split=random] X_train=(861, 60, 5), X_test=(216, 60, 5)
Loaded from cache: data_cache\CBA.AX_2020-01-01_2024-07-02.csv
NaN before: 0
NaN after : 0
Raw df shape: (1137, 5)
Clean df shape: (1137, 5)
Train/Test samples: 846 231
X_train shape (samples, lookback, n_features): (846, 60, 5)
Scaler keys: ['feature_scaler', 'target_scaler', 'feature_columns', 'target_column']
Feature columns: ['Open', 'High', 'Low', 'Close', 'Volume']
Target column: Close
```

3) Explanation of less straightforward lines of code (with reasoning)

3.1 Local caching (save/load CSV to reduce repeated downloads)

I needed to confirm the correct way to store a pandas DataFrame locally and reload it while preserving the date index. I also needed to design a consistent cache filename keyed by ticker and date range.

Key lines & reasoning:

- os.makedirs(cache_dir, exist_ok=True)
  Ensures the cache directory exists so saving does not fail on the first run.

```
os.makedirs(cache_dir, exist_ok=True)
```

- cache_name = f"{company}_{start_date}_{end_date}.csv" (sanitized).
  Generates a unique cache file per dataset range.

```
cache_name = _safe_filename(f"{company}_{start_date}_{end_date}.csv")
```

- if os.path.exists(cache_path): df = pd.read_csv(cache_path, index_col=0, parse_dates=True)
  Loads the cached dataset.
  - index_col=0 restores the saved index column (dates).
  - parse_dates=True ensures the index becomes a proper DatetimeIndex again.

- Else branch downloads from yfinance and saves:
  - df.to_csv(cache_path)
  - and prints “Saved/Loaded” messages to prove caching works.

```
    if use_cache and (not force_download) and os.path.exists(cache_path):
        print("Loaded from cache:", cache_path)
        df = pd.read_csv(cache_path, index_col=0, parse_dates=True)
    else:
        df = yf.download(company, start=start_date, end=end_date, progress=False)
        df = _flatten_yfinance_columns(df)
        if use_cache:
            df.to_csv(cache_path)
            print("Saved to cache:", cache_path)

    if not isinstance(df.index, pd.DatetimeIndex):
        df.index = pd.to_datetime(df.index)

    return df
```

Why caching matters:
yfinance depends on network availability and can be slow. Caching makes experiments reproducible and faster, and prevents repeated download failures.

3.2 Flattening yfinance MultiIndex columns (critical in some environments)

Why this required research:

In some yfinance versions/configurations, returned columns can be MultiIndex, like ('Close', 'CBA.AX'), which breaks normal column selection and many plotting libraries.

Key logic explained:

- _flatten_yfinance_columns(df) converts MultiIndex → single-level columns
  Example mapping: ('Close', 'CBA.AX') → 'Close'

Why this matters:

If columns remain MultiIndex, selecting df["Close"] may fail or return unexpected objects. Flattening ensures stable processing and plotting.

```python
def _flatten_yfinance_columns(df):
    """
    yfinance sometimes returns MultiIndex columns like:
        ('Close','CBA.AX'), ('Open','CBA.AX'), ...
    But our code (and many plotting libs) expect flat columns:
        'Close', 'Open', 'High', 'Low', 'Volume'

    If df.columns is MultiIndex, we keep only the first level ('Close', 'Open', ...).
    """
    if isinstance(df.columns, pd.MultiIndex):
        df = df.copy()
        df.columns = [c[0] for c in df.columns]
    return df
```

3.3 NaN handling strategies (why it's done before scaling and sequencing)

Why this required research:

NaNs are common in financial time series (missing days, partial feeds, invalid values). I needed to decide practical strategies and implement them as options.

Key logic explained:

- df.replace([np.inf, -np.inf], np.nan)
  Converts invalid infinities to NaN so they are handled consistently.

```python
    # Replace +/-inf with NaN so it can handle all missing/invalid values consistently
    df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

- Strategy options:

- ffill_bfill: forward-fill then back-fill
- Ensures continuity for time series sequences.
- interpolate(method="time"): estimates missing values using time index
- drop: removes rows with missing values (simple but may shrink dataset)

```python
    # 2) Handle NaNs (Requirement b)
    # ----------------------------
    if nan_strategy == "drop":
        df = df.dropna()
    elif nan_strategy == "ffill":
        df = df.ffill()
    elif nan_strategy == "bfill":
        df = df.bfill()
    elif nan_strategy == "ffill_bfill":
        # Common time-series baseline:
        # - forward fill uses last known value
        # - back fill fixes NaNs at the start of the series
        df = df.ffill().bfill()
    elif nan_strategy == "interpolate":
        # Time interpolation can be helpful for small gaps; we still ffill/bfill as a safety net
        df = df.interpolate(method="time").ffill().bfill()
    else:
        raise ValueError("nan_strategy must be one of: drop, ffill, bfill, ffill_bfill, interpolate")

    # Final safety: remove any remaining NaN rows
    df = df.dropna()
```

Why preprocessing is better than letting the model handle NaNs:

Neural networks and scalers cannot learn meaningful parameters when NaNs are present; NaNs often propagate and break training (loss becomes NaN). Handling NaNs early ensures stable numeric input and consistent sequences.

3.4 Split methods (date / ratio / random) and preventing time leakage

Why this required research:

Time-series forecasting must avoid training on future data. Date-based splitting is safest, but the task requires supporting multiple methods.

Key logic explained:

- Date split:
  Find the row index where the split date occurs, and assign all earlier rows to training.

```python
    if split_method == "date":
        if split_date is None:
            raise ValueError("split_date must be provided when split_method='date'")
        split_ts = pd.to_datetime(split_date)
```

- This preserves temporal order and avoids leakage.
- Ratio split: split_idx = int(len(df) * train_ratio)
  Deterministic, still preserves order if you split by index.

```
    elif split_method == "ratio":
        split_idx = int(n * train_ratio)
        split_idx = max(split_idx, prediction_days + 1)
        split_idx = min(split_idx, n - 1)
```

- Random split:
  Implemented as an option for the task requirement, but I understand it may leak future context and is not ideal for forecasting. This is why it should only be used when explicitly selected.

```
    else:
        # For random split, we don't use split_idx; we split sequences later.
        split_idx = None
```

Evidence:

I ran all 3 split modes and printed X_train/X_test shapes for each.

3.5 Scaling features correctly (fit on train only + store scalers)

Why this required research:

Scaling must avoid data leakage: fitting scalers on the full dataset uses future statistics and gives unfair advantage. Also, predictions must be converted back to real prices.

Key logic explained:

- feature_scaler.fit(X_train_features)
  Fit scaler only on training feature values.

```
        if scale_features:
            x_scaler.fit(train_df[feature_columns].values)
            y_scaler.fit(train_df[[target_column]].values)
```

- feature_scaler.transform(full_feature_matrix)
  Apply same scaling to train and test.

```
            X_scaled = x_scaler.transform(df_clean[feature_columns].values)
            y_scaled = y_scaler.transform(df_clean[[target_column]].values).reshape(-1)
```

- Separate scaler for target column:
  - target_scaler.fit(train_close)

  ensures correct inverse-transform for price predictions.

Storing scalers in a dictionary:

I store:

- feature_scaler
- target_scaler
- feature_columns
- target_column

```
scalers = {
    "feature_scaler": x_scaler,
    "target_scaler": y_scaler,
    "feature_columns": feature_columns,
    "target_column": target_column,
}
```

This supports future access for:

- inverse-transforming predictions back to real price units, e.g. target_scaler.inverse_transform(pred_scaled)

3.6 Building LSTM sequences (3D shape)

Why this required research:

LSTM expects 3D input: (samples, timesteps, features). With multi-feature data, the last dimension becomes n_features.

Key logic explained:

- For each time index i, build one sample:
  - X[i] = features[i-lookback : i]
  - y[i] = target[i]
- Output shape becomes:
- X_train: (N_train, lookback, n_features)

Evidence:

My output shows X_train shape = (846, 60, 5) which confirms the model is now trained using OHLCV features (5 features).

```
X_train shape (samples, lookback, n_features): (846, 60, 5)
Scaler keys: ['feature_scaler', 'target_scaler', 'feature_columns', 'target_column']
Feature columns: ['Open', 'High', 'Low', 'Close', 'Volume']
```